Documentación Unity

Stephany Olaes

1. Descripción del juego

Mosquetero, con prioridad de cojer el máximo de dinero, en este caso monedas, para poder comprarse nueva vestimenta y armaduras.

2. Mecánicas implementadas

Obligatorias:

- Caminar y saltar entre plataformas
- Recolectar monedas
- Contador de monedas recolectadas
- Doble salto

Opcionales:

- Botón pausa, conjela el juego y se muestra menú de pausa
- Menú de pausa con tres opciones:
  - Resumir
  - Reiniciar, también se pone a 0 el contador de monedas recolectadas
  - Salir, redirige al menú principal del juego
- Menú principal con dos opciones:
  - Nuevo juego
  - Resumir juego
  - Número actual de monedas recolectadas

3. Scripts desarrollados

Caminar y salta entre plataformas. Salto = tecla W, Caminar = A/D o flechas

PlayerController.cs

```
void Update()
{
    horizontal = Input.GetAxis("Horizontal");      // Lee el eje horizontal (-1 izq, 0 nada, 1 der). A/D o flechas por defecto.

    if (Input.GetKeyDown(KeyCode.W)) jumpPressed = true;
    jumpHeld = Input.GetKey(KeyCode.W);

    if (horizontal > 0.01f)
    {
        Vector3 s = transform.localScale;
        s.x = Mathf.Abs(s.x);
        transform.localScale = s;
    }
    else if (horizontal < -0.01f)
    {
        Vector3 s = transform.localScale;
        s.x = -Mathf.Abs(s.x);
        transform.localScale = s;
    }

    animator.SetBool("run", Mathf.Abs(horizontal) > 0.01f);
}
```

Recolectar monedas

PlayerAttackController.cs

```
void Attack()
{
    if (playerAnimator != null) playerAnimator.SetTrigger("attack");
    StartCoroutine(DestroyCoinsDelayed());
}

1 reference
IEnumerator DestroyCoinsDelayed()
{
    yield return new WaitForSeconds(hitDelay);
    int destroyed = CoinManager.Instance.DestroyCoinsInRange(attackOrigin.position, attackRange);
    if (destroyed > 0)
    {
        GameManager.Instance.HitCoin(destroyed);
    }
}
```

CoinManager.cs

```
public int DestroyCoinsInRange(Vector2 center, float radius)
{
    int destroyed = 0;
    for (int i = spawnedCoins.Count - 1; i >= 0; i--)
    {
        if (spawnedCoins[i] == null)
        {
            spawnedCoins.RemoveAt(i);
            continue;
        }
        if (Vector2.Distance(spawnedCoins[i].transform.position, center) <= radius)
        {
            Destroy(spawnedCoins[i]);
            spawnedCoins.RemoveAt(i);
            destroyed++;
        }
    }
    return destroyed;
}
```

Contador de monedas recolectadas

PlayerStats.cs

```
1 reference
public int AddTotalPoints(int points)
{
    currentPoints += points;
    return currentPoints;
}

2 references
public int GetTotalPoints()
{
    return currentPoints;
}

2 references
public void ResetPoints()
{
    currentPoints = 0;
}
```

GameManager.cs

```
1 reference
public void HitCoin(int count = 1)
{
    int points = PlayerStats.Instance.AddTotalPoints(count);
    canvasManager.UpdateCoinPoints(points);
}
```

CanvasManager.cs

```
public void UpdateCoinPoints(int points)
{
    if (TextCoinUI != null) TextCoinUI.text = points.ToString();
}
```

Botón pausa. Conjela juego + menú visible

CanvasManager.cs

```
0 references
public void ClickPause()
{
    Time.timeScale = 0f;
    pauseButton.SetActive(false);
    pauseMenu.SetActive(true);
}
```

Menú pausa

Resumir

CanvasManager.cs

```
public void ClickResume()
{
    Time.timeScale = 1f;
    pauseButton.SetActive(true);
    pauseMenu.SetActive(false);
}
```

Reiniciar + reset contador

CanvasManager.cs

```
0 references
public void ClickRestart()
{
    Time.timeScale = 1f;
    if (PlayerStats.Instance != null) PlayerStats.Instance.ResetPoints();
    SceneManager.LoadScene(SceneManager.GetActiveScene().name);
}
```

PlayerStats.cs

```
2 references
public void ResetPoints()
{
    currentPoints = 0;
}
```

Salir al menú principal

CanvasManager.cs

```
public void ClickQuit()
{
    SceneManager.LoadScene(0);
}
```

Menú principal

Nuevo juego

MainMenuController.cs

```
public void NewGame()
{
    SceneManager.LoadScene(1);          // Carga la escena del juego
    PlayerStats.Instance.ResetPoints();
}
```

Reusumir Juego

MainMenuController.cs

```
public void Continue()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(1);
}
```

Número actual de monedas recolectadas (mostrado en menú)

PlayerStats.cs

```
void Awake()
{
    if (Instance == null)
    {
        Instance = this;
        DontDestroyOnLoad(this.gameObject);
    }
    else
    {
        Destroy(gameObject);
    }
}
```

CanvasManager.cs

```
void Start()
{
    Time.timeScale = 1f;
    pauseButton.SetActive(true);
    pauseMenu.SetActive(false);
    if (PlayerStats.Instance != null)
    {
        UpdateCoinPoints(PlayerStats.Instance.GetTotalPoints());
    }

}
```

4. Problemas surgidos

- La parte del salto se queda encallada a veces. El personaje para de saltar y se queda en posición de salto al estar caminando.